

第10章: デプロイと総まとめ

おめでとうございます！最終章です。この章では、ローカル（自分のPC）で動いているアプリをインターネット上に公開して、誰でもアクセスできるようにします。

この章で行うこと

作業	内容	所要時間の目安
ビルド	Build：開発用コードを本番環境用に最適化する。ファイルサイズの圧縮やコードの最適化が行われる	5分
デプロイ	Deploy：最適化したコードをサーバーに配置して公開する。「あなたのアプリにURLが付いて、世界中からアクセスできる状態になる」こと	10～15分
本番設定	Supabaseのセキュリティ設定の確認と調整	10分
振り返り	この教材で学んだことの総まとめ	10分

デプロイとは？ 自分のPCで動いているアプリは、他の人からはアクセスできません。デプロイとは、アプリをインターネット上のサーバーに配置して、URLでアクセスできるようにすることです。レストランで例えると、自宅で試作した料理を実店舗で提供開始する段階です。

この章では、**Vercel**（ヴァーセル：Next.jsを開発している会社が提供するホスティングサービス。GitHubと連携するだけで自動デプロイできる）を使ってデプロイします。そして、この教材全体を通じて学んだことを振り返り、今後の学習ロードマップ（次に何を学ぶべきかの道筋）を示します。

目次

- 0. 前提知識: ビルド／デプロイ／環境変数とは
 - 1. デプロイの準備
 - 2. Vercel へのデプロイ
 - 3. Supabase の本番設定
 - 4. パフォーマンス最適化
 - 5. 学習の振り返り

- 6. 次のステップ（発展学習）
- 7. 推奨学習リソース
- 8. 付録: 全ファイルの完成版一覧
- 9. おわりに

0. 前提知識: ビルド／デプロイ／環境変数とは

0.1 ビルド（Build）とは

「書いたコードを、本番で実行できる形に変換する」処理です。`npm run build` で実行します。具体的には:

1. TypeScript → JavaScript に変換（ブラウザは TS を読めない）
2. JSX → 通常のJS関数呼び出しに変換
3. ファイルを圧縮（不要な空白・コメント削除、変数名を短縮）
4. ファイルを結合（バンドル）してリクエスト数を削減
5. 画像やフォントを最適化

▼ ビルド実行例:

```
$ npm run build
```

▼ 期待される出力（抜粋）:

```
> next build

  ▲ Next.js 15.0.0
  - Environments: .env.local

  Creating an optimized production build ...
  ✓ Compiled successfully in 12.3s
  Linting and checking validity of types
  ✓ Generating static pages (5/5)
  Finalizing page optimization

Route (app)                Size      First Load JS
┌ ○ /                      1.32 kB    92.5 kB
├ ○ /books                  2.45 kB    93.6 kB
├ λ /books/[id]             1.78 kB    92.9 kB
└ λ /books/[id]/edit        2.10 kB    93.2 kB

○ (Static)   prerendered as static content
λ (Dynamic)  server-rendered on demand
```

▼ 何が起こった？ - すべてのページが正常にビルドされた - 各ページのファイルサイズが表示される - ○ は静的ページ、λ はリクエストごとにサーバーで生成されるページ

ビルド失敗が出たら？: TypeScriptの型エラーや lint エラーが残っているとビルドできません。エラーメッセージのファイル名と行番号を見て修正します。**ローカルでビルドが通ってからデプロイする習慣**をつけると安全です。

0.2 デプロイ（Deploy）とは

ビルドの成果物をインターネット上のサーバーに配置して、誰でも URL でアクセスできるようにする作業です。本書では Vercel に GitHub 経由で自動デプロイします。

```
[あなたのPC]
  ↓ git push
[GitHub]                                ← コードの保管庫
  ↓ Webhookで自動通知
[Vercel]
  ↓ npm run build を実行
[CDN（世界中のサーバー）]              ← 完成品が配置される
  ↓
あなたのアプリ: https://your-app.vercel.app
```

0.3 環境変数（Environment Variables）とは

「コードに直接書きたくない設定値」を別の場所で管理する仕組みです。代表例：

変数名（例）	用途	機密性
NEXT_PUBLIC_SUPABASE_URL	Supabase のURL	公開可（ブラウザに送られる）
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase の匿名キー	公開可
SUPABASE_SERVICE_ROLE_KEY	Supabaseの管理者キー	絶対秘密（サーバー専用）

ローカル: `.env.local` ファイルに書く（`.gitignore` でGit管理から外れている）

```
# .env.local（このファイルはGitHubに上げない！）
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGc...
```

本番（Vercel）: Vercelダッシュボードの Settings → Environment Variables で同じ値を設定する。

NEXT_PUBLIC_ の意味: 接頭辞 `NEXT_PUBLIC_` を付けた環境変数は**ブラウザ側にも送られる**（＝世界に公開される）。付けない変数はサーバーでしか参照できないので、シークレットには `NEXT_PUBLIC_` を付けてはいけません。

0.4 開発・本番の3つの違い

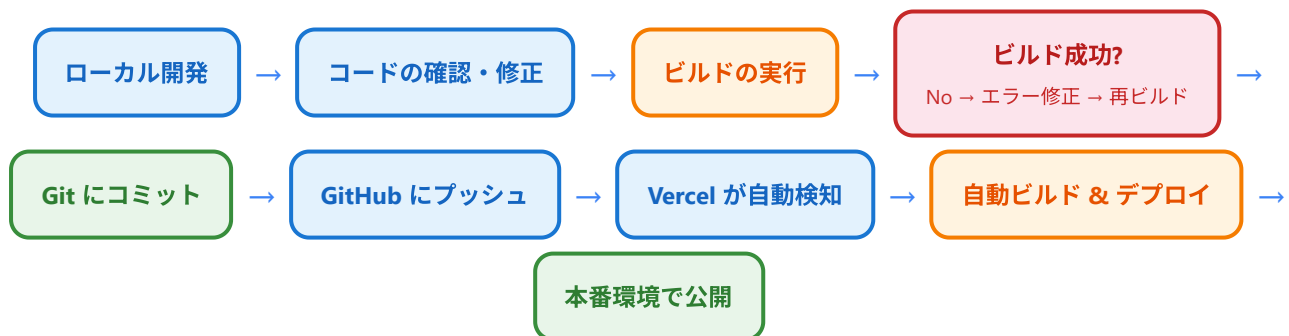
項目	開発 (npm run dev)	本番 (npm run build → vercel)
URL	<code>http://localhost:3000</code>	<code>https://your-app.vercel.app</code>
速度	遅い (毎回コンパイル)	速い (最適化済み)
エラー表示	詳細なスタックトレース	ユーザーには非表示
ホットリロード	あり (保存で即反映)	なし
ファイルサイズ	大きい	小さい (圧縮済み)
環境変数の出元	<code>.env.local</code>	Vercel の設定画面

1. デプロイの準備

アプリケーションを本番環境にデプロイする前に、いくつかの準備が必要です。ローカル環境では動いていても、本番環境では動かないケースがあるため、事前にしっかり確認しましょう。

1.1 デプロイまでの全体フロー

まず、開発からデプロイまでの全体的な流れを確認しましょう。



1.2 ビルドの実行

Next.js アプリケーションをデプロイする前に、必ずローカルでビルドを実行して問題がないか確認します。

```
# プロジェクトのルートディレクトリで実行
npm run build
```

ビルドが成功すると、以下のような出力が表示されます。

Route (app)	Size	First Load JS
└ ○ /	5.2 kB	89.4 kB
└ ○ /books	3.1 kB	87.3 kB
└ ○ /books/[id]	2.8 kB	87.0 kB
└ ○ /books/new	4.5 kB	88.7 kB
└ ○ /books/[id]/edit	4.7 kB	88.9 kB
+ First Load JS shared by all	84.2 kB	
└ chunks/framework-XXXXX.js	45.2 kB	
└ chunks/main-XXXXX.js	31.8 kB	
└ other shared chunks (total)	7.2 kB	
○ (Static) prerendered as static content		

ビルドとは？ 開発中のコード（TypeScript、JSX など）をブラウザが直接理解できる形式（JavaScript、HTML、CSS）に変換し、最適化する処理のことです。ビルドによってファイルサイズが小さくなり、読み込み速度が向上します。

1.3 よくあるビルドエラーと解決方法

ビルド時にエラーが出ることはよくあります。以下は、初心者がよく遭遇するエラーとその解決方法です。

エラー1: TypeScript の型エラー

```
Type error: Property 'title' does not exist on type 'Book'.
```

原因: 型定義と実際のコードが一致していない。

解決方法:

```
// 型定義を確認して修正する
type Book = {
  id: string;
  title: string;    // ← このプロパティが定義されているか確認
  author: string;
  rating: number;
  created_at: string;
};
```

エラー2: import エラー

```
Module not found: Can't resolve '@/components/BookCard'
```

原因: ファイルパスが間違っている、またはファイルが存在しない。

解決方法:

```
# ファイルの存在を確認
ls src/components/BookCard.tsx

# ファイル名の英文字・小文字も確認 (Linux ではケースセンシティブ)
# BookCard.tsx と bookCard.tsx は別のファイルとして扱われます
```

エラー3: 環境変数が undefined

```
Error: supabaseUrl is required.
```

原因: 環境変数が設定されていない、または `.env.local` が読み込まれていない。

解決方法:

```
# .env.local ファイルが存在するか確認
cat .env.local

# 以下の変数が設定されているか確認
# NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
# NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIs...
```

エラー4: 'use client' ディレクティブの不足

```
Error: useState only works in Client Components. Add the "use client" directive.
```

原因: Client Component のフック（`useState`、`useEffect` など）を Server Component で使っている。

解決方法:

```
// ファイルの先頭に追加
'use client';

import { useState } from 'react';
// ...
```

エラー5: ESLint エラー

```
ESLint: 'variable' is defined but never used. (@typescript-eslint/no-unused-vars)
```

原因: 使用されていない変数やインポートがある。

解決方法:

```
// 不要な変数・インポートを削除する
// または、意図的に未使用の場合はアンダースコアを付ける
const _unusedVariable = 'something';
```

1.4 環境変数の確認

デプロイ前に、必要な環境変数がすべて揃っているか確認しましょう。

```
# .env.local の内容を確認
cat .env.local
```

必要な環境変数一覧:

変数名	説明	例
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase プロジェクトの URL	<code>https://abcdefg.supabase.co</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase の匿名キー	<code>eyJhbGciOiJIUzI1NiIs...</code>

重要: `.env.local` は `.gitignore` に含まれているため、GitHub にはプッシュされません。これはセキュリティ上正しい動作です。デプロイ先（Vercel）で別途環境変数を設定する必要があります。

1.5 ビルド前チェックリスト

デプロイ前に以下をすべて確認しましょう。

- ☐ `npm run build` がエラーなく成功する
- ☐ `npm run lint` がエラーなく成功する
- ☐ `.env.local` に必要な環境変数がすべて設定されている
- ☐ `.gitignore` に `.env.local` と `node_modules` が含まれている
- ☐ ブラウザで全ページが正常に表示される
- ☐ CRUD 操作（作成・読取・更新・削除）がすべて動作する
- ☐ コンソールにエラーが出ていない

2. Vercel へのデプロイ

2.1 Vercel とは

Vercel（ヴァーセル）は、Next.js を開発している会社が提供するホスティングプラットフォームです。Next.js アプリケーションのデプロイに最適化されており、最も簡単かつ確実にデプロイできます。

Vercel を選ぶ理由:

- Next.js の開発元が運営しているため、最新機能への対応が最速
- GitHub と連携して、プッシュするだけで自動デプロイ
- 無料プラン（Hobby）で個人プロジェクトには十分な機能
- グローバル CDN で高速配信
- SSL（HTTPS）が自動で設定される
- プレビューデプロイ（Pull Request ごとにプレビュー URL を自動生成）

2.2 ホスティングサービスの比較

サービス	Next.js 対応	無料プラン	自動デプロイ	難易度	特徴
Vercel	最適	あり	あり	簡単	Next.js 公式。最も相性が良い
Netlify	良好	あり	あり	簡単	静的サイトに強い。Next.js 対応も改善中
AWS Amplify	良好	あり（制限付き）	あり	やや難	AWS エコシステムとの統合が強み
Railway	良好	あり（制限付き）	あり	普通	バックエンドも含めた総合プラットフォーム
Cloudflare Pages	部分的	あり	あり	やや難	エッジコンピューティングに強い

この教材では Vercel を使用します。Next.js との相性が最も良く、設定が最も簡単だからです。

2.3 事前準備: GitHub リポジトリの作成

Vercel にデプロイするには、コードが GitHub に存在する必要があります。まだ GitHub リポジトリを作成していない場合は、以下の手順で作成します。

Step 1: Git の初期化とコミット

```
# -----  
# プロジェクトのルートディレクトリ (package.json があるフォルダ) で実行する  
# -----  
  
# (1) Git リポジトリを初期化 (既に init 済みなら不要、何度実行してもOK)  
#     カレントフォルダに隠しフォルダ .git/ が作られ、Git管理が始まる  
git init  
# ▼ 出力例  
# Initialized empty Git repository in /path/to/book-management/.git/  
  
# (2) ステージングエリアに全ファイルを登録  
#     . はカレントフォルダ全体を意味する。  
#     .gitignore に書かれたファイル/フォルダ (node_modules や .env.local 等) は  
#     自動的に除外されるので安心。  
git add .  
  
# (3) 最初のコミット  
#     -m "... " はコミットメッセージ。何をした履歴か後で分かるよう短文で残す。  
git commit -m "書籍管理アプリの初回コミット"  
# ▼ 出力例  
# [main (root-commit) abc1234] 書籍管理アプリの初回コミット  
# 150 files changed, 12345 insertions(+)  
# create mode 100644 README.md  
# create mode 100644 package.json  
# ...
```

Step 2: GitHub にリポジトリを作成

1. [GitHub](#) にログインする
2. 右上の「+」ボタンをクリック → 「New repository」を選択
3. リポジトリ名を入力 (例: `book-management-app`)
4. 「Private」を選択 (公開したくない場合)
5. 「Create repository」をクリック

Step 3: ローカルリポジトリと GitHub を接続

```
# -----  
# GitHub のリポジトリ作成画面に表示される3行のコマンド  
# -----  
  
# (1) リモート接続先を「origin」という名前で登録  
#     URL は GitHub 上のリポジトリの URL に置き換える  
#     これ以降「origin」と書けばそのURLを指すようになる  
git remote add origin https://github.com/あなたのユーザー名/book-management-app.git  
  
# (2) ローカルのブランチ名を「main」に統一する（古い環境では master の場合がある）  
#     -M は「強制リネーム」を意味する  
git branch -M main  
  
# (3) 初回プッシュ  
#     -u origin main は「これ以降、git push だけで origin の main に送るよう覚えてね」  
#     という設定（upstream の設定）。  
git push -u origin main  
# ▼ 出力例  
# Enumerating objects: 150, done.  
# Counting objects: 100% (150/150), done.  
# ...  
# To github.com:yourname/book-management-app.git  
# * [new branch]      main -> main  
# branch 'main' set up to track 'origin/main'.
```

2.4 Vercel アカウントの作成

1. [Vercel の公式サイト](#) にアクセス
2. 「Sign Up」をクリック
3. 「Continue with GitHub」を選択して GitHub アカウントでログイン
4. Vercel が GitHub へのアクセス権限を要求するので「Authorize」をクリック
5. アカウント作成完了

注意: GitHub アカウントでログインすることで、リポジトリとの連携が簡単になります。

2.5 デプロイ手順（Step by Step）

Step 1: 新規プロジェクトの作成

1. Vercel ダッシュボード（<https://vercel.com/dashboard>）にアクセス
2. 「Add New...」→「Project」をクリック
3. 「Import Git Repository」セクションで、先ほど作成した `book-management-app` リポジトリを見つける

4. 「Import」をクリック

Step 2: プロジェクトの設定

インポート画面で以下を確認します。

- **Project Name:** `book-management-app` (変更可能)
- **Framework Preset:** `Next.js` (自動検出されるはず)
- **Root Directory:** `./` (プロジェクトのルート)
- **Build Command:** `npm run build` (デフォルトのまま)
- **Output Directory:** `.next` (デフォルトのまま)

Step 3: 環境変数の設定

これが最も重要なステップです。環境変数を設定しないと、Supabase に接続できずアプリが動作しません。

1. 「Environment Variables」セクションを展開する
2. 以下の変数を追加する

Key	Value
<code>NEXT_PUBLIC_SUPABASE_URL</code>	<code>https://あなたのプロジェクトID.supabase.co</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	<code>eyJhbGciOiJIUzI1NiIs...</code> (あなたの匿名キー)

値の確認方法: Supabase ダッシュボード → Settings → API で確認できます。

3. 「Add」ボタンで各変数を追加

Step 4: デプロイの実行

1. 「Deploy」ボタンをクリック
2. Vercel がビルドとデプロイを開始する (通常 1〜3 分)
3. 成功すると、紙吹雪のアニメーションとともにデプロイ完了画面が表示される
4. 表示された URL (例: `https://book-management-app.vercel.app`) をクリックしてアプリを確認

2.6 デプロイの自動化フロー

一度設定すれば、以降は GitHub にプッシュするだけで自動デプロイされます。



2.7 環境変数の追加・変更方法（デプロイ後）

デプロイ後に環境変数を変更したい場合:

1. Vercel ダッシュボードでプロジェクトを選択
2. 「Settings」タブ → 「Environment Variables」
3. 変数を追加・編集・削除
4. **重要:** 環境変数を変更した後は、「Deployments」タブから最新のデプロイを「Redeploy」する必要があります

2.8 カスタムドメインの設定（任意）

独自ドメイン（例: `mybooks.example.com`）を設定したい場合:

1. Vercel ダッシュボードでプロジェクトを選択
2. 「Settings」タブ → 「Domains」
3. ドメイン名を入力して「Add」
4. 表示される DNS 設定をドメインレジストラ（お名前.com、Google Domains など）で設定
5. DNS の反映を待つ（通常数分～最大48時間）

初心者へ: カスタムドメインは必須ではありません。Vercel が自動生成する `xxx.vercel.app` の URL でも十分に利用できます。

2.9 デプロイ後の動作確認

デプロイが完了したら、以下の項目を確認しましょう。

- ☐ トップページが正常に表示される
- ☐ 書籍一覧が Supabase から取得されて表示される
- ☐ 新しい書籍を追加できる
- ☐ 書籍の詳細ページが表示される
- ☐ 書籍の情報を編集できる
- ☐ 書籍を削除できる
- ☐ スマートフォンでも正常に表示される（レスポンス）
- ☐ HTTPS（鍵アイコン）で接続されている

トラブルシューティング: 問題が発生した場合は、Vercel ダッシュボードの「Deployments」→ 該当デプロイ → 「Logs」でビルドログとランタイムログを確認できます。

3. Supabase の本番設定

アプリが本番環境で公開された後は、セキュリティとデータ保護をしっかりと確認する必要があります。

3.1 セキュリティの確認

API キーの種類を理解する

Supabase には2種類のキーがあります。

キーの種類	用途	公開してよいか
<code>anon</code> キー（匿名キー）	クライアントサイドからのアクセス	はい（RLS で保護）
<code>service_role</code> キー	サーバーサイドからの管理アクセス	絶対に公開してはいけない

<code>NEXT_PUBLIC_SUPABASE_URL</code>	→ クライアントに公開される（OK）
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	→ クライアントに公開される（OK、RLS が必須）
<code>SUPABASE_SERVICE_ROLE_KEY</code>	→ サーバーサイドのみで使用（絶対に公開しない）

重要: `NEXT_PUBLIC_` プレフィックスが付いた環境変数は、ブラウザの JavaScript から参照可能です。
`service_role` キーには絶対に `NEXT_PUBLIC_` を付けないでください。

フロントエンドのコードで確認すべきこと

ブラウザの開発者ツール（F12 → Network タブ）で、以下を確認しましょう。

- Supabase への API リクエストが HTTPS で行われていること
- `service_role` キーがリクエストヘッダに含まれていないこと
- レスポンスに不要な個人情報が含まれていないこと

3.2 RLS（Row Level Security）ポリシーの見直し

RLS は Supabase のセキュリティの要です。本番環境では、適切なポリシーが設定されていることを必ず確認しましょう。

現在の RLS 設定を確認する

Supabase ダッシュボード → Table Editor → `books` テーブル → 「RLS」タブで確認できます。

この教材では、認証なしで誰でもアクセスできる設定にしています。

```
-- 現在のポリシー（開発用・学習用）
-- すべてのユーザーが読み取り可能
CREATE POLICY "誰でも書籍を読める" ON books
  FOR SELECT USING (true);

-- すべてのユーザーが書き込み可能
CREATE POLICY "誰でも書籍を追加できる" ON books
  FOR INSERT WITH CHECK (true);

-- すべてのユーザーが更新可能
CREATE POLICY "誰でも書籍を更新できる" ON books
  FOR UPDATE USING (true);

-- すべてのユーザーが削除可能
CREATE POLICY "誰でも書籍を削除できる" ON books
  FOR DELETE USING (true);
```

本番環境向けの推奨設定

本格的なアプリケーションでは、認証を導入した上で以下のようなポリシーに変更することを推奨します。

```

-- 本番用ポリシー（認証導入後）
-- 誰でも読み取り可能（公開データの場合）
CREATE POLICY "誰でも書籍を読める" ON books
  FOR SELECT USING (true);

-- 認証済みユーザーのみ書き込み可能
CREATE POLICY "認証済みユーザーのみ追加可能" ON books
  FOR INSERT WITH CHECK (auth.role() = 'authenticated');

-- 自分が追加した書籍のみ更新可能
CREATE POLICY "自分の書籍のみ更新可能" ON books
  FOR UPDATE USING (auth.uid() = user_id);

-- 自分が追加した書籍のみ削除可能
CREATE POLICY "自分の書籍のみ削除可能" ON books
  FOR DELETE USING (auth.uid() = user_id);

```

この教材の範囲: 認証機能は次のステップとして紹介しますので、現時点では開発用のポリシーのままで問題ありません。ただし、**重要なデータを扱う本番アプリではセキュリティ設定を必ず強化してください。**

3.3 バックアップの設定

データベースのバックアップは、データ損失を防ぐために非常に重要です。

Supabase の自動バックアップ

- **Free プラン:** 自動バックアップなし（手動でのみ可能）
- **Pro プラン以上:** 毎日の自動バックアップ、Point-in-Time Recovery（PITR）

手動バックアップの方法

Supabase ダッシュボードから手動でバックアップを取得できます。

1. Supabase ダッシュボード → Settings → Database
2. 「Database Backups」セクション
3. 「Download backup」をクリック

または、`pg_dump` コマンドを使用する方法:

```

# Supabase のデータベース URL は、ダッシュボードの
# Settings → Database → Connection string で確認できます

pg_dump "postgresql://postgres:[パスワード]@db.[プロジェクトID].supabase.co:5432/postgres" >

```

初心者へ: 学習目的であれば手動バックアップで十分です。本番アプリケーションを運用する際は、Pro プラン以上の利用を検討してください。

4. パフォーマンス最適化

デプロイしたアプリのパフォーマンスを向上させるためのテクニックを紹介します。

4.1 Next.js の Image コンポーネント

Next.js には、画像を自動的に最適化する `Image` コンポーネントが用意されています。書籍のカバー画像などを扱う場合に効果的です。

通常の `<img>` タグとの違い

```
// ❌ 通常の img タグ（最適化なし）


// ✅ Next.js の Image コンポーネント（自動最適化）
import Image from 'next/image';

<Image
  src="/book-cover.jpg"
  alt="書籍カバー"
  width={200}
  height={300}
  placeholder="blur"           // 読み込み中にぼかし表示
  blurDataURL="data:..."     // ぼかし画像のデータURL
/>
```

Image コンポーネントのメリット:

機能	説明
自動リサイズ	デバイスに合わせた最適なサイズで配信
形式変換	WebP/AVIF など効率的な形式に自動変換
遅延読み込み	ビューポートに入るまで読み込みを遅延
CLS 防止	width/height の指定でレイアウトシフトを防止
キャッシュ	最適化した画像をキャッシュして再利用

外部画像を使用する場合

外部サイトの画像を使用する場合は、`next.config.js` に許可するドメインを追加する必要があります。

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'example.com', // 画像を取得するドメイン
        pathname: '/images/**',
      },
      {
        protocol: 'https',
        hostname: '*.supabase.co', // Supabase Storage を使う場合
        pathname: '/storage/**',
      },
    ],
  },
};

module.exports = nextConfig;
```

4.2 メタデータの設定（SEO）

検索エンジンにアプリを正しく認識してもらうために、メタデータを設定しましょう。

ルートレイアウトでのメタデータ設定

```
// src/app/layout.tsx
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: {
    default: '書籍管理アプリ',
    template: '%s | 書籍管理アプリ', // 各ページのタイトルが自動的に「ページ名 | 書籍管理アプリ」に
  },
  description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
  keywords: ['書籍管理', '本', 'レビュー', 'Next.js'],
  authors: [{ name: 'あなたの名前' }],
  openGraph: {
    title: '書籍管理アプリ',
    description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
    url: 'https://your-app.vercel.app',
    siteName: '書籍管理アプリ',
    locale: 'ja_JP',
    type: 'website',
  },
  twitter: {
    card: 'summary_large_image',
    title: '書籍管理アプリ',
    description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
  },
  robots: {
    index: true,
    follow: true,
  },
};
```

各ページでのメタデータ設定

```
// src/app/books/page.tsx
import type { Metadata } from 'next';

export const metadata: Metadata = {
  title: '書籍一覧', // → 「書籍一覧 | 書籍管理アプリ」と表示される
  description: '登録されている書籍の一覧を表示します。',
};
```

動的なメタデータ（書籍詳細ページ）

```
// src/app/books/[id]/page.tsx
import type { Metadata } from 'next';

type Props = {
  params: { id: string };
};

export async function generateMetadata({ params }: Props): Promise<Metadata> {
  // Supabase から書籍データを取得
  const { data: book } = await supabase
    .from('books')
    .select('*')
    .eq('id', params.id)
    .single();

  return {
    title: book?.title ?? '書籍詳細',
    description: `${book?.title} (${book?.author}) の詳細情報`,
  };
}
```

4.3 Lighthouse での計測方法

Google Lighthouse は、Web アプリのパフォーマンス、アクセシビリティ、SEOなどを総合的に評価するツールです。

Lighthouse の使い方

1. Chrome ブラウザでデプロイしたアプリを開く
2. F12 キーで開発者ツールを開く
3. 上部タブから「Lighthouse」を選択
4. 「Analyze page load」をクリック
5. 数十秒後に結果が表示される

評価項目

項目	説明	目標スコア
Performance	ページの読み込み速度	90 以上
Accessibility	アクセシビリティ（音声読み上げ対応など）	90 以上
Best Practices	Web 開発のベストプラクティス準拠	90 以上
SEO	検索エンジン最適化	90 以上

スコアを上げるコツ

- **Performance:** Image コンポーネントの使用、不要な JavaScript の削除、フォントの最適化
 - **Accessibility:** alt 属性の設定、十分なカラーコントラスト、セマンティックな HTML
 - **Best Practices:** HTTPS の使用、コンソールエラーの解消
 - **SEO:** メタデータの設定、title タグの設定、レスポンシブデザイン
-

5. 学習の振り返り

ここまでお疲れさまでした。この教材を通じて、あなたは多くのことを学びました。一つずつ振り返ってみましょう。

5.1 この教材で学んだこと

以下のチェックリストで、あなたの学習成果を確認してください。

TypeScript の基礎

- [x] 変数の型注釈 (`string` , `number` , `boolean`)
- [x] 型エイリアス (`type Book = { ... }`)
- [x] ジェネリクスの基本的使用
- [x] `interface` と `type` の違い
- [x] オプションルプロパティ (`?`)
- [x] 型推論の仕組み

React の基礎

- [x] コンポーネントとは何か
- [x] JSX の書き方
- [x] Props の受け渡し
- [x] State の管理 (`useState`)
- [x] 副作用の処理 (`useEffect`)
- [x] イベントハンドリング (`onClick` , `onChange` , `onSubmit`)
- [x] 条件付きレンダリング
- [x] リストのレンダリング (`map` と `key`)
- [x] フォームの作成と制御

Next.js の基礎

- [x] App Router の仕組み (ファイルベースルーティング)
- [x] Server Components と Client Components の違い

- [x] `'use client'` ディレクティブの使い方
- [x] `page.tsx` と `layout.tsx` の役割
- [x] 動的ルート (`[id]`)
- [x] リンクとナビゲーション (`Link` コンポーネント)
- [x] メタデータの設定

Supabase でのデータベース操作

- [x] Supabase プロジェクトの作成
- [x] テーブルの作成
- [x] RLS (Row Level Security) の設定
- [x] データの取得 (`SELECT`)
- [x] データの追加 (`INSERT`)
- [x] データの更新 (`UPDATE`)
- [x] データの削除 (`DELETE`)
- [x] Supabase クライアントの初期化

CRUD アプリケーションの実装

- [x] Create: 新しい書籍の追加フォーム
- [x] Read: 書籍一覧の表示、詳細ページ
- [x] Update: 書籍情報の編集フォーム
- [x] Delete: 書籍の削除 (確認ダイアログ付き)
- [x] ローディング状態の処理
- [x] エラーハンドリング

Tailwind CSS でのスタイリング

- [x] ユーティリティクラスの基本
- [x] レスポンシブデザイン (`sm:` , `md:` , `lg:`)
- [x] Flexbox / Grid レイアウト
- [x] ホバー・フォーカスのスタイル
- [x] カラーパレットの使い方
- [x] カスタムコンポーネントのスタイリング

Vercel へのデプロイ

- [x] GitHub リポジトリの作成
- [x] Vercel アカウントの作成と連携
- [x] 環境変数の設定

- [x] 自動デプロイの仕組み
- [x] デプロイ後の動作確認

5.2 技術スタック全体像

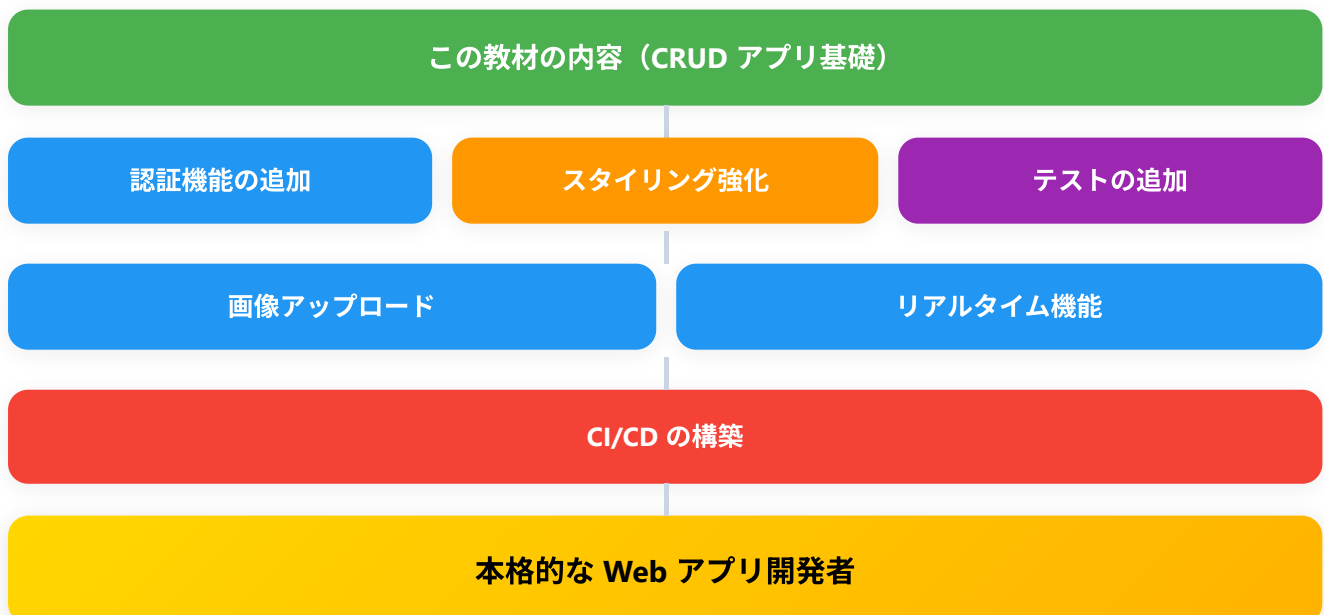
この教材で使用した技術スタックを図で確認しましょう。



6. 次のステップ（発展学習）

この教材で基礎を身につけたあなたは、さらに多くの機能を追加できます。以下に、次のステップとして取り組める発展的なトピックを紹介します。

6.1 学習ロードマップ



6.2 認証機能の追加（Supabase Auth）

ユーザーがアカウントを作成し、ログインしてから書籍を管理できるようにします。

実装のイメージ:

```
// Supabase Auth を使ったログイン例
import { supabase } from '@/lib/supabase';

// メールアドレス + パスワードでサインアップ
const { data, error } = await supabase.auth.signUp({
  email: 'user@example.com',
  password: 'your-password',
});

// ログイン
const { data, error } = await supabase.auth.signInWithPassword({
  email: 'user@example.com',
  password: 'your-password',
});

// Google ログイン
const { data, error } = await supabase.auth.signInWithOAuth({
  provider: 'google',
});

// ログアウト
const { error } = await supabase.auth.signOut();
```

学べること: - ユーザー認証の仕組み - セッション管理 - 保護されたルート（ログインしていないとアクセスできないページ） - RLS ポリシーとの連携

参考リソース: - [Supabase Auth ドキュメント](#) - [Next.js + Supabase Auth ガイド](#)

6.3 画像アップロード（Supabase Storage）

書籍のカバー画像をアップロードして表示する機能を追加します。

実装のイメージ:

```
// Supabase Storage を使った画像アップロード例
import { supabase } from '@lib/supabase';

// 画像のアップロード
const { data, error } = await supabase.storage
  .from('book-covers') // バケット名
  .upload(`covers/${fileName}`, file, {
    cacheControl: '3600',
    upsert: false,
  });

// アップロードした画像の公開 URL を取得
const { data: { publicUrl } } = supabase.storage
  .from('book-covers')
  .getPublicUrl(`covers/${fileName}`);
```

学べること: - ファイルアップロードの仕組み - Supabase Storage の使い方 - 画像のプレビュー表示 - ファイルサイズ・形式のバリデーション

参考リソース: - [Supabase Storage ドキュメント](#)

6.4 リアルタイム機能（Supabase Realtime）

複数ユーザーが同時にアプリを開いているとき、誰かが書籍を追加・編集すると、他のユーザーの画面にもリアルタイムで反映される機能です。

実装のイメージ:


```
// Supabase Realtime を使ったリアルタイム購読例
'use client';

import { useEffect } from 'react';
import { supabase } from '@lib/supabase';

export default function BooksPage() {
  useEffect(() => {
    // books テーブルの変更をリアルタイムで監視
    const channel = supabase
      .channel('books-changes')
      .on(
        'postgres_changes',
        {
          event: '*', // INSERT, UPDATE, DELETE すべて
          schema: 'public',
          table: 'books',
        },
        (payload) => {
          console.log('変更を検知:', payload);
          // ここで State を更新してUIに反映
        }
      )
      .subscribe();

    // クリーンアップ
    return () => {
      supabase.removeChannel(channel);
    };
  }, []);

  return <div>...</div>;
}
```

学べること: - WebSocket の仕組み - リアルタイムデータ同期 - 楽観的更新 (Optimistic Update)

参考リソース: - [Supabase Realtime ドキュメント](#)

6.5 テストの追加 (Jest, React Testing Library)

コードの品質を保証するために、テストを書く方法を学びます。

実装のイメージ:

```
// BookCard コンポーネントのテスト例
import { render, screen } from '@testing-library/react';
import BookCard from '@components/BookCard';

describe('BookCard', () => {
  const mockBook = {
    id: '1',
    title: '吾輩は猫である',
    author: '夏目漱石',
    rating: 5,
    created_at: '2025-01-01',
  };

  it('書籍のタイトルが表示される', () => {
    render(<BookCard book={mockBook} />);
    expect(screen.getByText('吾輩は猫である')).toBeInTheDocument();
  });

  it('著者名が表示される', () => {
    render(<BookCard book={mockBook} />);
    expect(screen.getByText('夏目漱石')).toBeInTheDocument();
  });

  it('評価が星で表示される', () => {
    render(<BookCard book={mockBook} />);
    const stars = screen.getAllByText('★');
    expect(stars).toHaveLength(5);
  });
});
```

学べること: - ユニットテスト・統合テストの考え方 - Jest テストランナーの使い方 - React Testing Library でのコンポーネントテスト - テスト駆動開発 (TDD) の基礎

参考リソース: - [Jest 公式ドキュメント](#) - [React Testing Library 公式ドキュメント](#)

6.6 CI/CD の構築

GitHub Actions を使って、コードをプッシュするたびに自動でテストを実行し、品質を保つ仕組みを構築します。

実装のイメージ:

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - run: npm ci
      - run: npm run lint
      - run: npm run build
      - run: npm test
```

学べること: - CI/CD の概念と重要性 - GitHub Actions の使い方 - 自動テスト・自動デプロイのパイプライン - ブランチ戦略 (main, develop, feature ブランチ)

参考リソース: - [GitHub Actions 公式ドキュメント](#)

7. 推奨学習リソース

7.1 公式ドキュメント

リソース名	URL	説明
Next.js 公式ドキュメント	https://nextjs.org/docs	Next.js のすべての機能を網羅。チュートリアルも充実
React 公式ドキュメント	https://ja.react.dev	React の基礎から応用まで。日本語版あり
TypeScript 公式ドキュメント	https://www.typescriptlang.org/docs/	TypeScript の型システムを深く学べる
Supabase 公式ドキュメント	https://supabase.com/docs	Supabase の全機能のガイド
Tailwind CSS 公式ドキュメント	https://tailwindcss.com/docs	ユーティリティクラスの全リファレンス
Vercel 公式ドキュメント	https://vercel.com/docs	デプロイ・設定の詳細ガイド
MDN Web Docs	https://developer.mozilla.org/ja/	Web 技術全般のリファレンス。日本語版あり

7.2 おすすめの書籍

書籍名	対象	概要
『プロを目指す人のためのTypeScript入門』	初級～中級	TypeScript を体系的に学べる定番書
『React ハンズオンラーニング 第2版』	初級～中級	React の基礎を手を動かしながら学べる
『実践 Next.js』	中級	App Router を含む Next.js の実践的な解説
『Web API: The Good Parts』	中級	API 設計の基礎を学べる
『リアクト! TypeScript で始めるつらくない React 開発』	初級	React + TypeScript を優しく解説

7.3 コミュニティ

コミュニティ	URL	特徴
Zenn	https://zenn.dev	日本語の技術記事プラットフォーム。Next.js 関連の記事が豊富
Qiita	https://qiita.com	日本最大級の技術情報共有サービス
Stack Overflow	https://stackoverflow.com	世界最大の Q&A サイト。英語だが情報量が圧倒的
Discord（各技術の公式サーバー）	-	Next.js、Supabase 等それぞれ公式 Discord がある
X（旧 Twitter）	https://x.com	<code>#nextjs</code> <code>#supabase</code> などのハッシュタグで最新情報をキャッチ

8. 付録: 全ファイルの完成版一覧

この教材で作成したすべてのファイルの一覧です。

8.1 プロジェクト構成

```
book-management-app/
├─ public/
│  └─ favicon.ico
├─ src/
│  └─ app/
│     │ layout.tsx
│     │ page.tsx
│     │ globals.css
│     └─ books/
│        │ page.tsx
│        │ new/
│        │ │ page.tsx
│        │ └─ [id]/
│        │   │ page.tsx
│        │   └─ edit/
│        │     │ page.tsx
│  └─ components/
│     │ BookCard.tsx
│     │ BookForm.tsx
│     │ Header.tsx
│     │ DeleteButton.tsx
│     └─ StarRating.tsx
├─ lib/
│  └─ supabase.ts
├─ types/
│  └─ book.ts
├─ .env.local
├─ .gitignore
├─ next.config.js
├─ package.json
├─ tailwind.config.ts
├─ tsconfig.json
└─ README.md
```

8.2 各ファイルの概要

ファイルパス	種類	説明
<code>src/app/layout.tsx</code>	レイアウト	アプリ全体の共通レイアウト。Header を含む
<code>src/app/page.tsx</code>	ページ	トップページ。アプリの紹介と書籍一覧へのリンク
<code>src/app/globals.css</code>	スタイル	Tailwind CSS のベーススタイル
<code>src/app/books/page.tsx</code>	ページ	書籍一覧ページ。Supabase からデータ取得して表示
<code>src/app/books/new/page.tsx</code>	ページ	新規書籍追加ページ。BookForm を使用
<code>src/app/books/[id]/page.tsx</code>	ページ	書籍詳細ページ。動的ルートでIDに基づく表示
<code>src/app/books/[id]/edit/page.tsx</code>	ページ	書籍編集ページ。既存データの更新
<code>src/components/BookCard.tsx</code>	コンポーネント	書籍カードの表示。一覧ページで使用
<code>src/components/BookForm.tsx</code>	コンポーネント	書籍の追加・編集フォーム。新規追加と編集で共用
<code>src/components/Header.tsx</code>	コンポーネント	ヘッダーナビゲーション
<code>src/components/DeleteButton.tsx</code>	コンポーネント	削除ボタン。確認ダイアログ付き
<code>src/components/StarRating.tsx</code>	コンポーネント	星評価の表示・入力コンポーネント
<code>src/lib/supabase.ts</code>	ユーティリティ	Supabase クライアントの初期化
<code>src/types/book.ts</code>	型定義	Book 型の定義
<code>.env.local</code>	環境変数	Supabase の接続情報（Git管理外）
<code>next.config.js</code>	設定	Next.js の設定ファイル
<code>tailwind.config.ts</code>	設定	Tailwind CSS のカスタム設定
<code>tsconfig.json</code>	設定	TypeScript のコンパイラ設定

9. おわりに

この教材を完走したあなたへ

この教材を最後まで読み進め、書籍管理アプリを完成させたあなたに、心からの祝福を送ります。

プログラミングの学習は、決して簡単な道のりではありません。新しい概念が次々と登場し、エラーに悩まされ、「自分には向いていないのかも」と思うこともあったかもしれません。それでも、ここまでたどり着いたということは、あなたには確実に「学び続ける力」があるということです。

振り返ってみてください。この教材を始めた時点のあなたと、今のあなたは全く違います。

- **TypeScript** で型安全なコードが書けるようになりました
- **React** でインタラクティブな UI を構築できるようになりました
- **Next.js** でモダンな Web アプリケーションを作れるようになりました
- **Supabase** でデータベースを操作できるようになりました
- **Tailwind CSS** で美しいデザインを実装できるようになりました
- **Vercel** にデプロイして、世界中からアクセスできるアプリを公開できるようになりました

これらはすべて、現在のWeb開発の現場で実際に使われている技術です。あなたは、実務で通用するスキルの基礎をすでに手に入れています。

大切なこと

プログラミング学習で最も大切なのは、**手を動かし続ける**ことです。

- この教材のアプリに新しい機能を追加してみてください
- 別のアプリのアイデアを考えて、ゼロから作ってみてください
- エラーに出会ったら、それは学びのチャンスです
- わからないことがあれば、公式ドキュメントを読む習慣をつけてください
- 完璧を求めず、まず動くものを作り、少しずつ改善していくことが大切です

「できない」は「まだできない」

今の時点で理解しきれていないことがあっても、心配はいりません。「できない」のではなく、「まだできない」だけです。プロのエンジニアも、毎日新しいことを学び続けています。学習に終わりはありませんが、それこそがプログラミングの面白さでもあります。

あなたが作った書籍管理アプリは、あなたのポートフォリオの第一歩です。これからもっと多くのアプリを作り、スキルを磨いていってください。

フィードバック

この教材に関するご意見・ご感想・誤りの報告は、以下の方法でお寄せください。

- **GitHub Issues**: このリポジトリの Issues ページで報告
- **Pull Request**: 修正提案がある場合は PR を作成

皆さまのフィードバックが、この教材をより良いものにします。

お疲れさまでした。あなたの Web 開発の旅は、ここから始まります。